A planar graph without four- or nine-cycles that is not three-colorable

P. Graham
https://orcid.org/0009-0007-8493-1505

October 20, 2026

Abstract

Lu, Wang, Wang, Bu, Montassier and Raspaud asked which integers $i \in \{5, 6, 7, 8, 9\}$ have the property that every planar graph without cycles of length four or i admits a proper three-coloring. To our knowledge, the first counterexample to the statement that every planar graph without four- or nine-cycles is three-colorable is given here: an 82-vertex planar graph with 136 edges and no four- or nine-cycles that is not three-colorable. An explicit tree expresses the graph as 27 copies of K_4 joined by 26 Ore compositions. The elementary fact that Ore composition preserves non-three-colorability gives an inductive proof requiring no coloring search. Explicit three-colorings of all 136 single-edge deletions certify four-criticality. As an immediate consequence, it attains the Kostochka–Yancey edge bound and has the fewest edges among all 82-vertex four-critical graphs; no minimum-order claim is made. A standalone checker replays the tree and checks the embedding, forbidden cycles and deletion colorings. Planarity is certified by a rotation system, and cycle exclusion by finite enumeration.

The proofs, the Lean formalization, the search code, the certificate checkers and the text were produced by AI models (from OpenAI and Anthropic), working under the author's direction.

Status: computer-assisted; not expert-reviewed; AI-assisted.

1 Introduction

All graphs in this paper are finite and simple. A proper three-coloring is a map $f : V(G) \to \{0, 1, 2\}$ with $f(u) \neq f(v)$ for every edge uv. A cycle is simple; it need not be induced. Thus forbidding a cycle of length k, denoted C_k, forbids cycles with chords as well.

Steinberg conjectured in 1976 that every planar graph without C_4 or C_5 is properly three-colorable; see the account in [4]. Lu, Wang, Wang, Bu, Montassier and Raspaud [1] posed a broader forbidden-pair question, as reproduced in Kang, Jin and Wang's Problem 1.2 [2]: for which $i \in \{5, 6, 7, 8, 9\}$ is every $\{C_4, C_i\}$-free planar graph properly three-colorable? The $i = 5$ case is Steinberg's conjecture. Cohen-Addad, Hebdige, Král', Li and Salgado disproved that case in a preprint posted in April 2016, published in 2017 [4]. Jin, Kang, Liu and Wang's 2020 preprint [3] explicitly describes the cases $i \in \{6, 7, 8, 9\}$ as open. To our knowledge, the first counterexample to the assertion that every planar graph without cycles of length four or nine is properly three-colorable is the graph given here.

Theorem 1. *There exists an 82-vertex planar graph with 136 edges, without cycles of length four or nine, that is four-critical and is not properly three-colorable. It is a four-Ore graph and has the fewest edges among all 82-vertex four-critical graphs.*

This is a negative answer to the explicitly posed $i = 9$ case. The particular graph also excludes C_{10}, an additional checked property recorded in the cycle table below. It contains C_5, C_6, C_7 and C_8, so it gives no information on $\{C_4, C_5, C_9\}$- or $\{C_4, C_8, C_9\}$-free planar graphs, nor on the cases $i \in \{6, 7, 8\}$. No assertion about minimum order is made. The non-colorability argument below is elementary; planarity is certified by a rotation system and cycle exclusion by finite enumeration.

The earlier construction does not itself give this exclusion: its G_1 component in Figure 1 [4] contains the nine-cycle $a, f, k, n, m, j, i, h, g, a$ in the paper's own labels. This component is retained in the published construction.

The distinction from neighboring positive theorems matters. Kang, Jin and Wang prove proper three-colorability when C_6 is also excluded [2]. The $\{C_4, C_7, C_9\}$-free class is properly three-colorable by Lu and coauthors, as also recorded in [2]; Kang, Jin and Zhu prove DP-3-colorability of that class [5]. DP-3-colorability implies three-choosability, which implies ordinary proper three-colorability. Consequently, failure of a list or DP coloring property alone would not supply the obstruction required here. Defective colorings are a different notion: a (d_1, d_2, d_3)-coloring permits each color class to induce maximum degree at most d_j. The $(1, 1, 0)$ result of Dai, Wang and Xu [6] and the $(2, 0, 0)$ result of Chen, Dai, Nie, Wang and Yu [7] concern the exact forbidden pair but allow monochromatic edges. They are compatible with Theorem 1. Xiao and Huang's weak DP-$(\mathcal{F}_2, \mathcal{F}_2, \mathcal{F}_0)$-coloring theorem for this pair uses classes $\mathcal{F}_d$ of forests of maximum degree at most d [8]; it likewise does not assert proper three-colorability.

The accessible question statement used here is the reproduction in [2]. The defective and weak DP statements cited above are read through publisher abstracts or excerpts rather than their full papers. The abstract of Ang–Leo's 2026 chapter [9] describes two nonplanar constructions, the second excluding lengths four, five and six; it supplies no planar counterexample to the assertion considered here. Its full text, the updated Bu–Wang–Wang survey *On Coloring Problems*, and Li–Niu–Tian's *On the strong Bordeaux Conjecture* remain unread [9, 10, 11]. These reading limits are retained in the priority assessment.

The supplied bundle contains the graph and rotation system reproduced in Appendix A, with the formula, certificate and checkers at the relative paths specified below. The two embedding and cycle implementations are separate Python and C programs. The Ore tree gives an elementary non-colorability proof. Two additional non-colorability checks are exact constraint elimination over the three color values and a DRAT certificate checked by DRAT-trim. The mathematical reductions and ordinary executable programs, including the CNF generator, remain part of the trust base for these executable routes. The separate Lean development is described in Section 6.

2 The graph and its embedding

The input `data/graph.txt`, reproduced in Appendix A, defines G on vertices $0, \ldots, 81$ by 136 unordered edges. The accompanying `data/rotation.txt` gives a cyclic order of the neighbors at each vertex. The two planarity implementations separately validate the edge count, simplicity, neighbor lists and connectivity.

Let $D = \{(u, v) : uv \in E(G)\}$ be the darts. Write $\rho_v(u)$ for the neighbor following u in the cyclic order at v. The face permutation is

$$\phi(u, v) = (v, \rho_v(u)).$$

Both implementations count 56 orbits of ϕ. To justify this test, place an oriented disk at every vertex, attach an untwisted band for each edge according to the cyclic orders, and cap each boundary

walk with a disk. The result is a connected closed orientable surface with a cellular embedding of G. The boundary walks are precisely the orbits of ϕ, so its Euler characteristic is

$$|V| - |E| + |F| = 82 - 136 + 56 = 2.$$

By the classification of closed orientable surfaces this surface is the sphere. Deleting a point outside the embedded graph gives a plane embedding. This proves planarity; no geometric drawing or numerical coordinate tolerance is involved.

3 Excluding cycles

The Python enumerator grows every simple path from its smallest vertex s, using only other vertices greater than s. It records a closure to s when the path has at least three vertices and its second vertex is smaller than its last. Every undirected simple cycle has a unique smallest vertex and exactly two orientations, exactly one of which meets this last test. Thus each cycle is counted once.

The C implementation instead chooses an edge (s, t) with $s < t$, and enumerates simple s–t paths avoiding that edge, with all other vertices greater than s. Closing by ts gives a simple cycle. A cycle whose smallest vertex is s is found twice, once for each of its two edges at s; division by two gives the count. The implementation checks that each raw count is even. Both implementations run through length 14 and return the same table:

length	3	4	5	6	7	8	9	10	11	12	13	14
count	27	0	22	55	44	11	0	0	64	320	832	1824

In particular there is no C_4, C_9 or C_{10}. Since the maximum degree of this input is 10, even the coarse bound $82 \cdot 10^{14} < 2^{63}$ on raw enumeration counts is below the signed 64-bit limit used in C. All Python counts use arbitrary precision integers.

4 An elementary Ore-composition proof

Take vertex-disjoint graphs A and B, an edge xy of A, and a vertex z of B. Partition $N_B(z)$ into two nonempty disjoint sets N_x, N_y. Delete xy from A, replace z in B by two nonadjacent vertices, and identify those vertices with x, y, respectively, assigning neighbors N_x to x and N_y to y. The resulting graph is an *Ore composition*, also called a Dirac–Hajós–Gallai–Ore (DHGO) composition [13].

Lemma 1. *If neither A nor B is three-colorable, their Ore composition is not three-colorable.*

Proof. Suppose the composite has a proper three-coloring f. Restrict f to $V(A)$. If $f(x) \neq f(y)$, restoring the deleted edge xy gives a proper three-coloring of A, a contradiction. Hence $f(x) = f(y)$. Restrict f to $V(B) \setminus \{z\}$ and give z this common color. Every neighbor of z belonged to N_x or N_y, and therefore has a color different from that common color. Edges not incident with z are unchanged. This is a proper three-coloring of B, again a contradiction. □

The explicit tree in `data/ore-tree.json`, also printed in Appendix B, has 27 leaves and 26 internal nodes. Each leaf is a K_4; each internal node specifies both operands, the deleted edge, the split vertex and its neighborhood partition. Nodes are numbered in postorder, so their operands have smaller numbers. Replay uses the vertex labels stated in the tree. Its root has vertex set $\{0, \ldots, 81\}$ and exactly the edge set in `data/graph.txt`; the stated isomorphism is the identity $v \mapsto v$ for every vertex. The standalone checker verifies four distinct labels and all six edges at

each leaf. At each join it checks that both operand indices are smaller than the current index, the operand vertex sets are disjoint, xy is an edge of the first operand, z is a vertex of the second, and N_x, N_y are nonempty and form an exact partition of $N_B(z)$. It then performs precisely the deletion, split and identifications just defined. Finally it checks root equality under the stated bijection and that every non-root node is used exactly once, so every node contributes to the induction.

The graph K_4 is not three-colorable: four pairwise adjacent vertices need four distinct colors. Inducting over this explicit tree with Lemma 1 proves that G is not three-colorable. This proof requires no search for colorings and no unsatisfiability certificate. The finite structural replay checks that the displayed construction produces the particular published graph. Starting with 27 copies of K_4, each join reduces the sum of orders and the sum of sizes by one, so the root has $27 \cdot 4 - 26 = 82$ vertices and $27 \cdot 6 - 26 = 136$ edges.

4.1 Four-criticality and the edge bound

A graph is *four-critical* if it has chromatic number four and every proper subgraph is three-colorable. For each of the 136 edges e, `data/deletion-colorings.json` supplies an explicit vector $(c_0, \ldots, c_{81}) \in \{0,1,2\}^{82}$ properly coloring $G - e$. The checker verifies every edge other than e, and exact coverage of all edges, without solving any coloring problem. The two endpoints of e necessarily have equal colors, since otherwise this would color G. Recoloring one endpoint with a fourth color yields a proper four-coloring of G. Together with the preceding induction, this proves $\chi(G) = 4$. The graph has no isolated vertices. Any proper subgraph omits some edge of G, even if it omits a vertex, and is therefore a subgraph of some $G - e$. Restricting that deletion coloring proves its three-colorability. Consequently G is four-critical.

A graph obtained from copies of K_4 by Ore compositions is called a *four-Ore graph* [13]; the tree proves this property directly. Kostochka and Yancey's bound [12], reproduced as Theorem 4 of the cited arXiv version of [13], states that every four-critical graph H satisfies

$$|E(H)| \geq \left\lceil \frac{5|V(H)| - 2}{3} \right\rceil.$$

For this graph,

$$136 = \frac{5 \cdot 82 - 2}{3}.$$

Thus, as an immediate consequence of the construction and the bound, 136 is the minimum possible number of edges of any 82-vertex four-critical graph, including graphs without a planarity or forbidden-cycle restriction. This is an edge minimum at fixed order. Every four-Ore graph of this order shares this edge count; no uniqueness is asserted. It does not assert that 82 is the minimum order of a planar four-chromatic graph without four- or nine-cycles. The checker verifies the equality arithmetic; the general lower bound is the cited theorem.

5 Two additional checks of non-colorability

5.1 Exact constraint elimination

For each edge uv start with the relation $R_{uv} = \{(a,b) \in \{0,1,2\}^2 : a \neq b\}$. Maintain a conjunction of relations on the remaining vertices. To eliminate v, collect all relations mentioning it, let S be the union of their scopes minus v, and replace them by the relation

$$R_S = \{a \in \{0,1,2\}^S : \exists c \in \{0,1,2\} \bigwedge_{R \text{ mentioning } v} R(a, v = c)\}.$$

All other relations remain. The new conjunction is exactly the existential projection of the old one. Induction on eliminations shows that satisfiability is preserved. An empty relation proves that no coloring exists; a nonempty relation on the empty scope is a tautology and may be discarded.

The implementation enumerates all $3^{|S|}$ assignments and all three values of c. It chooses a vertex of minimum degree in the current interaction graph, breaking ties by vertex label. This choice affects cost, not correctness. On the specified graph it reaches an empty relation; the largest remaining scope S encountered has size three. This route imports no SAT solver, CNF generator or DRAT checker.

5.2 Exact graph-to-CNF reduction and symmetry

Independently introduce Boolean variables $x_{v,c}$, numbered $3v+c+1$, for $0 \le v < 82$ and $c \in \{0,1,2\}$. The generator adds the clauses

$$\begin{aligned} &x_{v,0} \vee x_{v,1} \vee x_{v,2} && \text{for each } v, \\ &\neg x_{v,a} \vee \neg x_{v,b} && \text{for each } v \text{ and } 0 \le a < b < 3, \\ &\neg x_{u,c} \vee \neg x_{v,c} && \text{for each } uv \in E(G) \text{ and } c. \end{aligned}$$

The first two lines specify exactly one color at each vertex, and the third specifies distinct colors across every edge. Assigning the unique true color is therefore a bijection between satisfying assignments of these clauses and proper three-colorings of G.

The lexicographically first triangle is $(0,1,6)$. The generator also adds $x_{0,0}$, $x_{1,1}$ and $x_{6,2}$, that is, unit clauses 1, 5 and 21. Every proper coloring gives distinct colors to this triangle. There is a permutation of the three color names taking them, in this order, to $0,1,2$; the other clauses are invariant under this permutation. Adding these units thus preserves satisfiability. No graph automorphism or unproved assertion about the search is used.

The resulting formula has $3 \cdot 82 = 246$ variables and $4 \cdot 82 + 3 \cdot 136 + 3 = 739$ clauses. The supplied `data/col3.cnf` is compared with the generated formula by parsed header and ordered clauses, ignoring comments. The certificate `data/col3.drat`, generated with CaDiCaL [15], is checked by DRAT-trim [14]. It contains 97 addition/deletion steps in standard text DRAT, ending with the empty clause. Its ordered steps are a lossless transcription of the binary proof. DRAT's checked redundancy conditions preserve satisfiability under additions; deletions only weaken the formula. Derivation of the empty clause therefore proves unsatisfiability of the input. This permits replay without trusting a solver's bare unsatisfiability answer or depending on a solver version. Together with the symmetry argument and the exact reduction above, this separately proves that G has no proper three-coloring.

6 Replay, controls and scope

The standalone executable check is `sh checks/check.sh`, run from the bundle root. It needs Python 3.10 or later, a C compiler and a POSIX shell; there are no third-party Python packages or network downloads. It compiles the supplied C sources in a temporary directory, runs both embedding checks, both cycle enumerators, the Ore replay and all deletion colorings, elimination, CNF comparison and DRAT verification. The single standard-library script `checks/check_ore.py`, given the graph, tree, rotation and deletion-coloring files, also checks planarity, all simple cycles through length ten, the Ore induction and four-criticality on its own. It imports no other bundled code. The DRAT route requires successful termination and the exact success status, not just an absent error message.

The same command exercises deliberate corruptions. Deleting edge $(0, 1)$ must make the elimination result colorable and invalidate the original DRAT proof on the modified formula. Adding $(0, 12)$ creates the cycle $0, 6, 60, 12, 0$; adding $(0, 7)$ creates $0, 1, 6, 60, 12, 62, 63, 13, 7, 0$. Both cycle implementations must reject each modified graph. Swapping two neighbors in the rotation at zero changes the Euler characteristic to zero and must be rejected by both embedding checks. This is rejection of a rotation, not a non-planarity claim for the underlying graph. Finally a proper prefix containing only the first 45 complete proof steps must fail DRAT verification. A tenth control repeats a vertex in the first K_4 leaf of the tree; the standalone checker must reject it. These are ten control outcomes. The certificate checks and the reductions in the preceding sections establish Theorem 1.

The full statement of Theorem 1 remains computer-assisted. For the executable routes above, the rotation-system argument, enumeration completeness, Ore induction, criticality argument, CNF reduction, color symmetry and constraint-elimination argument are mathematical arguments written here. The Python and C implementations and DRAT-trim run as ordinary programs and are not themselves formally verified. The different implementations and non-colorability methods reduce shared implementation risk. The witness was found by composing small planar graphs with designated contact vertices, tabulating the possible color patterns on those contacts, and seeking a composition with no allowed pattern. This explains the discovery route; the certificate checks apply to the final edge list directly. The search is not needed for the proof and makes no claim of exhaustive search, optimal order or applicability to another forbidden pair.

Lean development. The Lean theorem `Steinberg.counterexample` conjoins only three properties of one `SimpleGraph (Fin 82)`: a spherical rotation certificate, no simple C_4 or C_9, and non-three-colorability. Lean parses the unchanged `data/graph.txt` and `data/rotation.txt` to define this graph and rotation. The planarity certificate is a fully checked connected genus-zero rotation, including exact neighbor incidence, cyclic vertex orders, exhaustive face orbits covering all darts, and $82 - 136 + 56 = 2$. The correspondence with a topological embedding uses the surface argument above and is not formalized in Lean. Cycle exclusion covers all Mathlib `IsCycle` closed walks, including cycles with chords. Non-colorability uses a proved coloring-to-CNF reduction and kernel-proved equality with the symmetry-free input of a verified RUP checker using LRAT-style hints. Deletion-erased RUP retains deleted clauses; every retained clause is proved entailed by the input. The Lean certificate constants are checked by kernel reduction; no SAT solver is trusted. Run `lake exe cache get` then `lake build` from `lean/`. The build guards the top statement's exact type and `#print axioms` using `#guard_msgs`, permitting only `propext`, `Classical.choice` and `Quot.sound`. No native/compiler proof evaluation or additional axioms are used. Four-criticality, the Ore decomposition, the edge minimum, C_{10} exclusion, and the full statement of Theorem 1 lie outside this Lean scope.

Credits and license. The Ore-composition method and the sharp edge bound are credited to the work discussed by Kostochka and Yancey [13]. Credit for posing and developing the forbidden-cycle questions belongs to the researchers cited above, and for the earlier Steinberg disproof to Cohen-Addad and coauthors. The formalization uses Lean 4 and its standard library and Mathlib; credit is due to their contributors. The hint-based certificate methodology is credited to Cruz-Filipe, Heule, Hunt, Kaufmann and Schneider-Kamp [16]. The SAT certificate uses CaDiCaL and is checked with Heule and Wetzler's DRAT-trim. This work, including the paper, is under Apache-2.0 (the repository's root `LICENSE`); the unmodified DRAT-trim source retains MIT. The full applicable notices and third-party provenance appear in `THIRD_PARTY.md`.

References

[1] H. Lu, Y. Wang, W. Wang, Y. Bu, M. Montassier and A. Raspaud. On the 3-colorability of planar graphs without 4-, 7- and 9-cycles. *Discrete Math.* 309 (2009), 4596–4607. doi:10.1016/j.disc.2009.02.030.

[2] Y. Kang, L. Jin and Y. Wang. The 3-colorability of planar graphs without cycles of length 4, 6 and 9. *Discrete Math.* 339 (2016), 299–307. arXiv:1506.04629 (first posted 2015).

[3] L. Jin, Y. Kang, P. Liu and Y. Wang. $(1,0,0)$-colorability of planar graphs without cycles of length 4 or 6. arXiv:2001.00166 (2020).

[4] V. Cohen-Addad, M. Hebdige, D. Král', Z. Li and E. Salgado. Steinberg's conjecture is false. *J. Combin. Theory Ser. B* 122 (2017), 452–456. arXiv:1604.05108 (first posted 2016).

[5] Y. Kang, L. Jin and X. Zhu. Planar graphs having no cycle of length 4, 7, or 9 are DP-3-colorable. *J. Graph Theory* 107 (2024), 344–358. doi:10.1002/jgt.23123; arXiv:2303.04657.

[6] L. Dai, Y. Wang and J. Xu. Every planar graph without cycles of length 4 or 9 is $(1,1,0)$-colorable. *Discrete Math.* 340 (2017), 2108–2122. doi:10.1016/j.disc.2017.04.006.

[7] M. Chen, L. Dai, J. Nie, Y. Wang and W. Yu. Planar graphs without cycles of length 4 or 9 are $(2,0,0)$-colorable. *Sci. Sin. Math.* 50 (2020), 317–338. doi:10.1360/N012018-00129.

[8] Y. Xiao and M. Huang. A weak DP-coloring of planar graphs without 4- and 9-cycles. *Discrete Appl. Math.* 342 (2024), 391–397. doi:10.1016/j.dam.2023.10.004.

[9] R. K. Ang and Z. H. Leo. Exploring the Weak Steinberg's Conjecture (2026). doi:10.1007/978-981-95-9630-0_37.

[10] Y. Bu, W. Wang and Y. Wang. On Coloring Problems. doi:10.1007/978-1-4614-6624-6_59-1.

[11] X. Li, L. Niu and F. Tian. On the strong Bordeaux Conjecture. *Discrete Math.* 349 (2026), 114818. doi:10.1016/j.disc.2025.114818.

[12] A. Kostochka and M. Yancey. Ore's Conjecture on color-critical graphs is almost true. *J. Combin. Theory Ser. B* 109 (2014), 73–101. arXiv:1209.1050; doi:10.1016/j.jctb.2014.05.002.

[13] A. Kostochka and M. Yancey. A Brooks-type result for sparse critical graphs. *Combinatorica* 38 (2018), 887–934. arXiv:1408.0846, version 3 (2017), Theorem 4 and the definition of Ore composition.

[14] N. Wetzler, M. J. H. Heule and W. A. Hunt, Jr. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. *SAT 2014*, LNCS 8561, 422–429. doi:10.1007/978-3-319-09284-3_31.

[15] A. Biere, K. Fazekas, M. Fleury and M. Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020. *Proc. SAT Competition 2020*, 51–53. CaDiCaL source and contributors.

[16] L. Cruz-Filipe, M. J. H. Heule, W. A. Hunt, Jr., M. Kaufmann and P. Schneider-Kamp. Efficient Certified RAT Verification. *CADE 2017*, LNCS 10395, 220–236. doi:10.1007/978-3-319-63046-5_14.

A Explicit graph and rotation

The graph list starts with the order and size, followed by every edge. The rotation gives every neighbor in cyclic order. These are the same input files used by the standalone check; no reconstruction from a drawing is necessary.

Graph

```
82 136
0 1
0 2
0 3
0 4
0 5
0 6
1 6
1 18
1 19
1 20
1 21
2 23
2 32
3 24
3 33
4 28
4 34
5 29
5 35
6 60
6 61
7 8
7 9
7 10
7 11
7 12
7 13
7 14
7 15
7 16
7 17
8 36
8 40
9 37
9 41
10 42
10 44
11 43
11 45
12 60
12 62
13 61
13 63
14 68
14 72
15 69
15 73
16 74
16 76
17 75
17 77
18 51
18 64
19 52
19 65
20 56
20 66
21 57
21 67
22 23
22 24
22 25
22 26
23 32
24 33
25 26
25 38
25 39
26 42
26 43
27 28
27 29
27 30
27 31
28 34
29 35
30 31
30 36
30 37
31 46
31 47
32 33
34 35
36 40
37 41
38 46
38 48
39 47
39 49
40 41
42 44
43 45
44 45
46 48
47 49
48 49
50 51
50 52
50 53
50 54
51 64
52 65
53 54
53 70
53 71
54 74
54 75
55 56
55 57
55 58
55 59
56 66
57 67
58 59
58 68
58 69
59 78
59 79
60 62
61 63
62 63
64 65
66 67
68 72
69 73
70 78
70 80
71 79
71 81
72 73
74 76
75 77
76 77
78 80
79 81
80 81
```

Rotation

```
0: 1 3 2 5 4 6
1: 0 6 19 18 21 20
2: 23 0 32
3: 33 0 24
4: 28 0 34
5: 35 0 29
6: 1 0 60 61
7: 12 8 9 11 10 14 15 17 16 13
8: 7 36 40
9: 37 7 41
10: 42 7 44
11: 45 7 43
12: 60 7 62
13: 7 61 63
14: 7 68 72
15: 69 7 73
16: 74 7 76
17: 77 7 75
18: 51 1 64
19: 65 1 52
20: 56 1 66
21: 67 1 57
22: 25 23 24 26
23: 22 2 32
24: 3 22 33
25: 38 22 26 39
26: 22 42 43 25
27: 30 28 29 31
28: 27 4 34
29: 5 27 35
30: 36 27 31 37
31: 27 46 47 30
32: 2 33 23
33: 32 3 24
34: 4 35 28
35: 34 5 29
36: 8 30 40
37: 30 9 41
38: 46 25 48
39: 25 47 49
40: 41 8 36
41: 9 40 37
42: 26 10 44
43: 11 26 45
44: 10 45 42
45: 44 11 43
46: 31 38 48
47: 39 31 49
48: 49 46 38
49: 47 48 39
50: 53 51 52 54
51: 50 18 64
52: 19 50 65
53: 70 50 54 71
54: 50 74 75 53
55: 58 56 57 59
56: 55 20 66
57: 21 55 67
58: 68 55 59 69
59: 55 78 79 58
60: 6 12 62
61: 13 6 63
62: 63 60 12
63: 61 62 13
64: 18 65 51
65: 64 19 52
66: 20 67 56
67: 66 21 57
68: 14 58 72
69: 58 15 73
70: 78 53 80
71: 53 79 81
72: 73 14 68
73: 15 72 69
74: 54 16 76
75: 17 54 77
76: 16 77 74
77: 76 17 75
78: 59 70 80
79: 71 59 81
80: 81 78 70
81: 79 80 71
```

B The explicit Ore tree

Each leaf row specifies the four vertices of a complete graph. Each join row specifies A, B, xy, z, N_x, N_y in the notation of Section 4. Labels at least 82 are temporary split vertices; none survives at the root, node 52. Operands are disjoint at every join. The root-to-published isomorphism is the identity on $0, \ldots, 81$.

node	vertices of K_4
0	$0, 1, 6, 7$
1	$12, 60, 62, 84$
2	$13, 61, 63, 85$
5	$50, 53, 54, 83$
6	$16, 74, 76, 89$
7	$17, 75, 77, 90$
10	$55, 58, 59, 88$
11	$14, 68, 72, 92$
12	$15, 69, 73, 93$
15	$20, 56, 66, 91$
16	$21, 57, 67, 94$
20	$70, 78, 80, 87$
21	$71, 79, 81, 95$
24	$18, 51, 64, 86$
25	$19, 52, 65, 96$
29	$22, 25, 26, 82$
30	$10, 42, 44, 100$
31	$11, 43, 45, 101$
34	$27, 30, 31, 99$
35	$8, 36, 40, 103$
36	$9, 37, 41, 104$
39	$4, 28, 34, 102$
40	$5, 29, 35, 105$
44	$38, 46, 48, 98$
45	$39, 47, 49, 106$
48	$2, 23, 32, 97$
49	$3, 24, 33, 107$

node	A	B	x	y	z	N_x	N_y
3	1	2	62	84	85	63	13, 61
4	0	3	6	7	84	60, 61	12, 13
8	6	7	76	89	90	77	17, 75
9	5	8	54	83	89	74, 75	16, 17
13	11	12	72	92	93	73	15, 69
14	10	13	58	88	92	68, 69	14, 15
17	15	16	66	91	94	67	21, 57
18	14	17	55	88	91	56, 57	20, 21
19	9	18	53	83	88	59	14, 15, 20, 21
22	20	21	80	87	95	81	71, 79
23	19	22	53	59	87	70, 71	78, 79
26	24	25	64	86	96	65	19, 52
27	23	26	50	83	86	51, 52	18, 19
28	4	27	1	7	83	18, 19, 20, 21	14, 15, 16, 17
32	30	31	44	100	101	45	11, 43
33	29	32	26	82	100	42, 43	10, 11
37	35	36	40	103	104	41	9, 37
38	34	37	30	99	103	36, 37	8, 9
41	39	40	34	102	105	35	5, 29
42	38	41	27	99	102	28, 29	4, 5
43	33	42	25	82	99	31	4, 5, 8, 9
46	44	45	48	98	106	49	39, 47
47	43	46	25	31	98	38, 39	46, 47
50	48	49	32	97	107	33	3, 24
51	47	50	22	82	97	23, 24	2, 3
52	28	51	0	7	82	2, 3, 4, 5	8, 9, 10, 11